

Question 1: Analysis of Returns, Risks, and Trading Strategies in the A-share Market

I. Problem Description

Stock prices are influenced by various factors, including market conditions, a company’s financial performance, and investors’ expectations. Simply observing daily or short-term price changes isn’t enough to fully assess a stock’s performance. A trading strategy that has historically yielded good results might merely happen to coincide with certain market conditions at that time. Therefore, when analyzing stocks, it’s necessary to consider factors like returns, volatility, drawdowns, trading patterns, and transaction costs. Additionally, it’s important to test the conclusions using data from periods when the strategy wasn’t actually used, to ensure their validity.

This study focuses on the daily price data of 20 A-share stocks from 2024 to 2025. Data from 2024 is used to understand stock characteristics, select indicators, and determine trading rules. Data from 2025 is utilized for out-of-sample testing. Required tasks include analyzing the entire stock pool, comparing returns and risks, designing trading signals, and conducting out-of-sample backtesting with cost considerations. All conclusions must be supported by tables, charts, and quantitative results.

The dataset contains a total of 9,700 daily market records. Each of the 20 stocks has 485 trading days. The following fields are provided:

- Stock code, stock name, trading date;
- Unadjusted opening prices, closing prices, highest prices, and lowest prices.
- The opening price, closing price, highest price, and lowest price after the price restoration.
- Trading volume, transaction value, post-adjusted price changes, volatility, and turnover rate.

The stock pool is as follows:

Stock Code	Stock Name	Stock Code	Stock Name
000001	Ping An Bank	600028	Sinopec
000333	Midea Group	600030	CITIC Securities
000651	Gree Electric	600036	China Merchants Bank
000858	Wuliangye	600050	China Unicom
002415	Hikvision	600276	Hengrui Medicine
002594	BYD	600519	Guizhou Moutai
300015	Aier Eye Hospital	600900	Yangtze Power
300059	Orient Fortune	601318	Ping An of China
300750	CATL	601398	Industrial and Commercial Bank of China

Stock Code	Stock Name	Stock Code	Stock Name
600009	Shanghai Airport	601899	Zijin Mining

II. Experimental Requirements

- [15 points] Organize the market data of 20 stocks and complete the overall statistics for the stock pool.
- [20 Points] Calculate the return and risk metrics for the years 2024 and 2025, as well as for the entire two-year period. Compare the differences between various stocks.
- [20 Points] Use data from 2024 to determine technical indicators and parameters, and establish clear rules for buying, selling, and holding positions.
- [25 points] Apply the finalized rules to an out-of-sample backtest for 2025. Include transaction costs in the analysis, and compare the results with those of a buy-and-hold strategy.
- [20 points] Design a GUI to manage stock data information.

III. Instructions for Required Parts

1. Data Organization and Overall Analysis (15 points)

(1) Overall statistics of the stock pool

Calculate the following indicators for all 20 stocks:

- Average daily trading volume over two years;
- Average daily trading volume over two years;
- Two-year average amplitude;
- Two-year average turnover rate;
- Proportion of trading days with price increases;
- Normalized index of back-adjusted closing prices.

The proportion of rising trading days is defined as the ratio of the number of valid trading days with a positive yield after price adjustment to the total number of valid yield records.

$$U_i = \frac{\#\{t: r_{i,t} > 0\}}{\#\{t: r_{i,t} \text{ 有效}\}} \times 100\%$$

The normalization index uses the post-adjusted closing price of each stock on the first trading day of 2024 as 100:

$$I_{i,t} = \frac{P_{i,t}}{P_{i,0}} \times 100$$

2. Revenue, Risk, and Stock Comparisons (20 points)

(1) Indicators and unified measurement standards

In this section, the adjusted closing price P is used consistently. For all 20 stocks, the following indicators are calculated for the years 2024 and 2025, as well as for the entire two-year period.

The daily simple rate of return is defined as:

$$r_{i,t} = \frac{P_{i,t}}{P_{i,t-1}} - 1$$

For each statistical period, the daily return is calculated using the prices of adjacent trading days within that period. The cumulative return for the period is calculated using the post-adjusted closing prices of the first and last trading days of that period.

$$R_i = \frac{P_{i,end}}{P_{i,start}} - 1$$

The annualized volatility is calculated based on the sample standard deviation of daily returns:

$$\sigma_i = s(r_i)\sqrt{243}$$

Maximum drawdown is calculated based on the cumulative net value: $V_{i,t}$

$$MDD_i = \max_t \left(1 - \frac{V_{i,t}}{\max_{s \leq t} V_{i,s}} \right)$$

The risk-adjusted metrics all use the annualized Sharpe ratio, with a risk-free rate of 0:

$$SR_i = \frac{\bar{r}_i}{s(r_i)}\sqrt{243}$$

Additionally, a stock pool with equal weights was constructed using the 20 stocks in the attachment as a reference. The daily return for reference purposes is defined as:

$$r_{EW,t} = \frac{1}{20} \sum_{i=1}^{20} r_{i,t}$$

Based on the daily return series, the cumulative returns, annualized volatility, maximum drawdown, and Sharpe ratio are calculated using an equal-weighted approach. This reference is entirely generated from the data in the attachment, with no need to obtain market indices separately.

3. Trend Indicators and Trading Signal Design (20 points)

(1) Indicator and rule design

Using only 2024 data, select one or a combination of indicators from existing fields such as moving averages, momentum, price breaks, volume changes, transaction value changes, amplitude, or turnover rate to create trading rules.

The rules must be clearly stated:

- The price criteria and indicator calculation formulas used;
- Metric period or threshold;
- The minimum number of trading days required before the indicator becomes effective.
- Buying conditions, selling conditions, and position status when there are no signals.
- How to handle conflicting signals on the same day;
- The relationship between the date when the signal is generated and the actual trading date.

If a moving average crossover is used, the short-term and long-term time periods should be specified. If momentum or breakout strategies are utilized, the backtesting period should be indicated. When volume, trading volume, or turnover rates are used, the comparison criteria and thresholds must be defined. Two sets of candidate parameters or two rules should be compared, and the final approach should be selected based on the results for 2024.

(2) Multiple stock verification

The final rules must generate signals based on the 2024 data for all 20 stocks, rather than simply showing the performance of the stock that did best afterward.

For each stock, at least the following should be recorded:

- Number of buy signals;
- Number of sell signals;
- Number of actual trading days held and their proportion of total trading days in 2024;
- The average number of days held within each position range has been calculated.

If a stock does not generate a signal or does not complete a trade, that result should be retained and explained. It is not allowed to force additional trades by modifying parameters.

4. Out-of-sample tests considering transaction costs (25 points)

(1) Backtesting scope

Maintain the indicators, parameters, and trading rules established in 2024 as applied to all 20 stocks in 2025. The following backtesting criteria will be uniformly used:

- Only long positions are taken, with no use of financing, leverage, or short selling.
- Each stock is allocated the same initial amount of capital, and its initial state is “empty position”.
- Each time a purchase is made, all available funds in the stock account are used to place the order. Each time a sale is made, all positions are closed. It’s possible to calculate transactions in fractional shares, without being restricted to trading in multiples of 100 shares.
- Calculate strategy returns using the adjusted price after the event.
- The calculation of indicators can use historical data from the end of 2024. If, based on information from the last trading day of 2024, the rules dictate that a position should be held on the next trading day, then a purchase should be made at the opening of the first trading day in 2025. Otherwise, remain in a cash position and wait for new buying signals.
- If the signal depends on information that becomes available only after the closing of the n th day, then the transaction will be executed at the reweighted opening price of the $(n+1)$ st trading day.
- A transaction fee of 0.10% of the transaction amount is charged at the time of purchase.
- When selling, a transaction fee of 0.10% of the transaction amount is charged, plus a 0.05% stamp duty.
- If there are still positions held on the last trading day of 2025, they should be sold at the reweighted closing price of that day, and the sale price should be recorded as part of the cost of sales.
- No additional assumptions are made regarding slip points that cannot be observed in the attachments.

The strategy portfolio of 20 stocks is combined by ensuring that each stock receives the same initial amount of capital, with no reallocation of funds between stocks. The net value of the portfolio is the arithmetic average of the net values of the 20 individual stock strategies.

For each stock, the same initial capital and cost assumptions as those used in the strategy are applied. Stocks are purchased on the first trading day of 2025 and sold on the last trading day of the year. The net value of holding all 20 stocks is calculated as the arithmetic average of the net values of holding each individual stock. No reallocation of funds between stocks or rebalancing is performed during the holding period.

(3) Expected Results

1. A comparison table of results from the 2024 sample versus those from the 2025 sample, showing the cumulative returns, volatility, maximum drawdowns, Sharpe ratios, number of trades, and costs associated with each strategy and holding period.
2. A table showing the backtesting results for all 20 stocks in 2025;
3. A curve showing the net value after fees for the 2025 synthetic strategy and the buy-and-hold approach.
4. A drawdown curve comparing the synthetic strategy for 2025 with a buy-and-hold approach.
5. A chart showing the ranking of 20 stocks in terms of “strategic return minus buy-and-hold return”.
6. A scatter plot showing the returns of strategies within the 2024 sample compared to those of strategies outside the 2025 sample.

(4) Result Evaluation

Will the stocks or parameters that performed well in 2024 continue to perform well in 2025?

Whether the strategy’s returns are primarily driven by a few individual stocks;

Do transaction costs significantly reduce the returns of a strategy?

The decline in out-of-sample performance may be due to parameter instability, excessive trading, signal lag, or changes in market conditions.

If the strategy doesn’t generate positive returns, what can the failure tell us?

Question 2: Spatiotemporal Analysis of Global M4.5 and Above Earthquakes

I. Problem Description

Seismological catalogs record information such as the time of earthquake occurrence, epicenter location, depth, magnitude, and measurement accuracy. By organizing these catalogs using specialized programs, it's possible to analyze the number of earthquakes over different time periods, the distribution of earthquakes across various magnitude and depth ranges, and their spatial distribution around the globe. However, seismological catalogs are continually updated based on new observations. There are also variations in the way magnitudes are determined, the coverage of seismic stations, and positioning errors. Therefore, when analyzing these data, it's important to perform calculations accurately and avoid interpreting the statistical results as predictions of future earthquakes or assessments of disaster risks.

This question uses data from the USGS ANSS Comprehensive Earthquake Catalog (ComCat). The search criteria are:

- The period covered is from January 1, 2024, 00:00:00.000 UTC to December 31, 2025, 23:59:59.999 UTC.
- `mag >= 4.5`;
- `eventtype=earthquake`;
- Output the CSV in ascending order by occurrence time.

The attachment contains a total of 14,953 records and the following 22 fields:

time, latitude, longitude, depth, mag, magType,
nst, gap, dmin, rms, net, id, updated, place, type,
horizontalError, depthError, magError, magNst, status,
locationSource, magSource

Among these, “time” refers to the time when the earthquake occurred, measured in UTC. “Updated” indicates the most recent time when this record was updated, not the time of the earthquake. The units for “depth”, “horizontalError”, and “depthError” are kilometers. “Gap” is measured in degrees. “dmin” represents the angular distance from the epicenter to the nearest station, also in degrees. “Rms” is measured in seconds. “Mag” is the preferred magnitude value used in this dataset. The file also includes values for mb, mww, mwr, ml, mw, mwb, and md. It's important not to refer to all these values uniformly as “Richter scale” or “moment magnitude”.

This problem requires using Python to perform directory verification, time statistics, classification by magnitude and depth, analysis of global latitude and longitude distributions, statistics within specified coordinate ranges, and analysis of representative events. “Place” refers merely to the named regions near an event; it isn't a strict representation of countries or administrative regions. Therefore, this problem does not require nor allow country-level statistics to be derived by splitting the “place” data.

II. Experimental Requirements

1. [20 points] Read the CSV file, properly distinguish between “time” and “updated” fields. Verify the range of records, their uniqueness, numerical boundaries, and check for any missing quality-related fields. Finally, export the processed data along with the verification results.

2. [20 points] Create a sequence of event counts over 24 consecutive months, and conduct overall and annual comparisons regarding magnitude, depth, and magType.
3. [30 minutes] Create a global latitude and longitude distribution map and a spatial statistical grid map. Implement a reusable coordinate-window statistical function, and compare three fixed windows. $10^\circ \times 10^\circ$
4. [10 Points] Create a list of the top 10 most significant events. Calculate the relative energy index on a monthly basis. Compare the “month with the highest number of events” with the “month with the highest relative energy index”.
5. [14 points] Optional: Design an interactive GUI tool for browsing and filtering earthquake catalogs.
6. [6 points] Optional: Choose any other machine learning method for data analysis.

The required section is worth 80 points, and the optional section is worth 20 points. For the required section, you must submit the Python source code that can be run directly, as well as the required CSV files, PNG images, and HTML files.

III. Instructions for Required Parts

1. Data reading, time organization, and directory verification (20 minutes)

(1) Read data and generate analysis fields

Using Pandas to read the entire attachment, parse the “time” and “updated” fields as times with UTC information. Sort the data by “time” and “id”. The following derived fields can be generated:

- Year: The year in which the event occurred, as determined by the time information.
- month: 由 time 得到的 YYYY-MM;
- mag_group: The magnitude classification specified in item 2.
- depth_group: The depth grouping specified in item 2.

All statistics for the 2024–2025 period must be based on the “time” parameter. Some of the records in the attachment have an “updated” date of 2026. This is a normal occurrence since the directory was revised after the earthquake. Records should not be deleted based on this information.

(2) Complete verifications that can be rechecked

The program should check whether any fields are missing, whether the data falls within reasonable ranges, and perform any necessary preprocessing and data cleaning.

In the attachment data, there are minor missing values in only six quality measurement fields: nst, gap, dmin, horizontalError, magError, and magNst. For calculations that involve these fields, any missing records for that field should be excluded from the calculation, and the actual sample size should be reported accordingly. Missing values should not be filled in with 0s, nor should any records with missing values be removed from all subsequent calculations in order to create “complete data”. The original table contains the non-zero value 0. This value should not be arbitrarily treated as a missing value without proper justification, nor should empty cells be filled in with 0s.

2. Time, Magnitude, and Depth Structure Analysis (20 points)

(1) Create a continuous monthly series

Count the number of events each month from January 2024 to December 2025, by month. The results must cover all 24 consecutive months. Even if the number of events in a particular month is 0, that month should still be included in the analysis. Calculate the total number of records per year, the average number of records

per month, the median, and the monthly maximum values. Also, identify the months with the highest and lowest numbers of events.

The monthly count can only be referred to as “the number of events in the currently frozen directory that meet the search criteria.” Two years of data is not sufficient to determine long-term trends in seismic activity, nor can it be used to predict future earthquakes.

(2) Layering by unified boundaries

The magnitude is defined using the following interval: left-closed, right-open.

[4.5, 5.0)、 [5.0, 6.0)、 [6.0, 7.0)、 [7.0, $+\infty$)

The following course statistics intervals are used in depth:

[0, 70) km、 [70, 300) km、 [300, $+\infty$) km

The above depth boundaries are used solely for uniform comparison across this problem. Using `pd.cut(..., right=False)` or equivalent methods, the following can be obtained:

- Table of the number of magnitude categories and their percentages by year;×
- Table showing the number of depth levels and their percentage by year;×
- Table of the quantity and proportion of magType;
- Cross-table of magnitude stratification and depth stratification.×

Different magType values mean different methods for calculating the magnitude. It’s possible to compare the distributions in the catalog, but different types should not be treated as completely identical precise measurements without further clarification.

This attachment contains 7,051 records with a depth of exactly 10 km. According to the USGS field specifications, depth is a parameter with relatively weak constraints in earthquake location determination. When observational data is insufficient, the location algorithm may use fixed depths such as 5 km, 10 km, or 33 km. Therefore, the depth statistics must include the number and percentage of records with a depth of 10 km. It should also be noted that this concentration of values may reflect the use of fixed depths, and should not be interpreted as indicating that the epicenters of all 7,051 earthquakes were accurately determined to be at a depth of 10 km.

3. Global Distribution, Spatial Grids, and Coordinate Window Functions (30 points)

(1) Plot the global latitude and longitude distribution and spatial grid

Create a global latitude/longitude scatter plot. The horizontal axis represents longitude, the vertical axis represents latitude. Color indicates depth, and point size indicates magnitude.

Divide the area into semi-open intervals at regular intervals along longitude and latitude. This creates a spatial counting matrix, which can then be used to create a heat map. The value in each cell represents the number of events that occurred within that specific longitude and latitude range. Since the actual area of cells at different latitudes varies, this graph should be referred to as a “spatial grid counting map,” not a “seismic density map per unit area.” $10^{\circ}18 \times 36$

Also generate an interactive `global_earthquakes.html` file. The interactive chart should display the id, time, mag, magType, depth, and place values when the user hovers over the elements. If ordinary latitude/longitude coordinates are used instead of an online base map, the title should clearly state “Latitude/Longitude Interactive Scatter Plot”.

Due to the large amount of data, only a portion can be plotted, but the number of records must not be less than 15.

(2) Implement coordinate window statistical functions

Implement the function:

```
summarize_box(df, lon_min, lon_max, lat_min, lat_max)
```

The function must check:

- $-180 \leq \text{lon_min} \leq \text{lon_max} \leq 180$;
- $-90 \leq \text{lat_min} \leq \text{lat_max} \leq 90$;
- The input data includes fields for longitude, latitude, time of occurrence, magnitude, and depth.

Ordinary windows use closed interval filtering and return one row of results, which must include at least:

```
lon_min, lon_max, lat_min, lat_max,  
event_count, count_2024, count_2025,  
count_mag_ge_6, max_mag, median_depth_km
```

Call the function for the statistical windows of the following three courses:

Window	Longitude range	Latitude range
A	[125,150]	[25,50]
B	[−85, −65]	[−60,15]
C	[90,145]	[−15,15]

These windows are merely used to verify the program and compare the rectangular coordinate ranges of spatial subsets. They do not represent national borders, political regions, or tectonic plate boundaries. Do not assign names of countries or tectonic plates to these windows without authorization.

4. Representative Events and Relative Energy Index (10 points)

Sort in descending order by mag, ascending order by time, and ascending order by id. Extract the top 10 events. Stable sorting rules must be implemented in the code; do not rely on the original row order of the data.

To compare the impact of extreme events on the aggregated results at logarithmic magnitudes, a dimensionless relative energy index is defined:

$$E_{\text{rel}} = 10^{1.5(M-4.5)}$$

Here, “mag” refers to the magnitude value in the attachments. The index for M4.5 earthquakes is defined as 1. This formula is merely an empirical approximation of the relationship between magnitude differences and energy differences over time. Since this dataset includes earthquakes of various “magType” values, the resulting value should be referred to as a “relative energy index” or “approximate proxy measure.” It cannot be considered as actual measured energy, precise joule values, intensity, degree of damage, or disaster losses.M

Calculate the number of events, the sum of relative energy indices, and the maximum magnitude for each month. Also, determine the proportion of the relative energy index of the strongest event in the total index over the two-year period.

IV. Instructions for Optional Sections

1. GUI-based earthquake catalog browsing and filtering tool

Design the program using Tkinter, PyQt, or other Python GUI libraries. Directly read the course-related CSV file, or the earthquakes_prepared.csv file generated as part of the required tasks. Ensure the following is implemented at minimum:

1. Select a file, load the data, and display the file name, number of records, time range, and loading status.
2. Filter by UTC date range, minimum magnitude, depth range, and magType. Include a reset function and accurately display the number of records after filtering.
3. The monthly quantity chart and latitude/longitude scatter plot can be updated based on the selected filters.
4. The table displays the filtered results. When a row is selected, the main fields of the event are displayed.
5. Export the current filter results as a CSV file, and save the current graph as a PNG file.

Referenced official materials:

- USGS Earthquake Catalog API: <https://earthquake.usgs.gov/fdsnws/event/1/>
- ComCat field description: <https://earthquake.usgs.gov/data/comcat/index.php>
- Explanation of the relationship between USGS earthquake magnitudes and energy: https://earthquake.usgs.gov/education/how_much_bigger.php
- Pandas read_csv: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
- Matplotlib: <https://matplotlib.org/stable/>
- Plotly Python: <https://plotly.com/python/>

Question 3: Given the time when vulnerabilities were exploited, information about the manufacturers, and analysis of the vulnerability structures

I. Problem Description

There are numerous vulnerabilities in both software and hardware. It's difficult to determine which vulnerabilities have actually been exploited in real-world attacks, based solely on vulnerability IDs or descriptions. The U.S. Cybersecurity and Infrastructure Security Agency (CISA) maintains the Known Exploited Vulnerabilities Catalog (KEV), which includes vulnerabilities that have a CVE ID, for which there are clear remediation instructions, and for which there is reliable evidence of actual exploitation in the wild. CISA uses this catalog as an important factor in determining the priority of vulnerability mitigation efforts. However, this catalog doesn't cover all vulnerabilities, nor does it provide information about how often a vulnerability has been exploited, how many devices are affected, or a unified severity score for each vulnerability.

This question uses the course attachment `CISA_KEV_2026-07-29.json`. It contains a total of 1,656 vulnerability records.

The top level of JSON contains:

title, catalogVersion, dateReleased, count, vulnerabilities

Each record in the vulnerabilities array contains the following 11 fields:

cveID, vendorProject, product, vulnerabilityName,
dateAdded, shortDescription, requiredAction, dueDate,
knownRansomwareCampaignUse, notes, cwes

Among them, cwes is a list. One CVE can correspond to multiple CWE codes, or the list can be empty. dateAdded represents the date when the vulnerability was added to the KEV directory. It's not the date the vulnerability was disclosed, the date an attack occurred, or the date the CVE number was assigned. dueDate refers to the deadline for taking required actions as specified in the CISA catalog. This isn't a universal deadline that all organizations must comply with, nor is it the actual date when repairs were completed. knownRansomwareCampaignUse can only take the values "Known" or "Unknown". "Unknown" indicates that CISA has not confirmed that this vulnerability has been exploited by ransomware. This should not be interpreted as "not used in ransomware activities".

This task requires using Python to parse nested JSON data. The required tasks include verifying the data structure, analyzing the time and expiration dates of entries, compiling statistics on manufacturers and their product combinations, analyzing the concentration of manufacturer-related tags, gathering statistics on CWE-related tags, and creating reusable filtering functions. All conclusions must be based solely on the data contained in the local KEV snapshot. It is not allowed to use the number of records to assess a manufacturer's product quality or the actual probability of being attacked.

II. Experimental Requirements

1. [20 points] Read the top-level metadata and vulnerability array from the JSON file. Verify the fields, CVE uniqueness, dates, enum values, and CWE lists. Then export the processed data.

2. [20 points] Analyze the monthly and annual distributions of dateAdded, the maturity structure of dueDate-dateAdded, and the confirmation status of ransomware in the Known and Unknown categories.
3. **【20 Points】** Calculate the number of records and their proportion for each manufacturer and manufacturer-product combination. Determine the concentration of manufacturer labels, and generate either a static image or an interactive HTML report.
4. [20 points] Correctly expand multi-value CWEs, compare the main CWEs, and implement query functions that can combine dates, manufacturers, ransomware status, and CWE conditions.
5. [14 points] Optional: Design a GUI for KEV directory retrieval and visualization.
6. [6 points] Optional: Use other machine learning methods to perform data analysis.

In all CSV files for this question, the percentage fields named share, cumulative_share, known_share, and unknown_share should be stored as decimals. These values should be formatted as percentages in the reports and charts. The HHI for manufacturers must also be calculated using these percentage values. Do not square the percentage values directly.[0,1][0,1]

III. Instructions for Required Parts

1. JSON reading, structure processing, and data validation (20 points)

(1) Read top-level metadata and the vulnerability array

First, use the Python standard library's json module to read the file. Then, use pd.json_normalize or an equivalent method to convert the vulnerabilities array into a DataFrame. The program outputs and verifies the result:

- catalogVersion;
- dateReleased;
- Top-level count;
- The actual length of the vulnerabilities;
- The set of original fields for each vulnerability record.

The top-level count must match the number of rows in the DataFrame. cwes should remain a list; it cannot be treated as a regular string during the reading process.

(2) Complete field and logical validation

Check:

1. Whether it contains 1,656 records and 11 original vulnerability fields.
2. Is the cveID not empty, unique, and meets the regular expression ^CVE-[0-9]{4}-[0-9]{4,19}\$?
3. Can both dateAdded and dueDate be parsed in the YYYY-MM-DD format?
4. Are all the conditions dueDate >= dateAdded met?
5. knownRansomwareCampaignUse 是否只包含 Known 和 Unknown;
6. Are all elements in cwes lists? And does each non-empty element satisfy the pattern ^CWE-[0-9]+\$?
7. Check whether there are any null values or empty strings for the manufacturer, product, vulnerability name, description, and required action.
8. The number of non-null values, the number of null values, and the data type for each field.

The CWES field contains 171 empty records in the attachment. All other main fields are complete. An empty CWE field indicates that no corresponding CWE codes are provided in the attachment. Do not fill in values like CWE-0, "Unknown", or common CWE codes. These records should be retained for tasks that do not involve CWE codes.

In addition, 6 records in vendorProject and 10 records in product contain leading or trailing spaces. The original fields must be retained. Additionally, vendor_clean and product_clean should be created, using .str.strip() to remove only leading and trailing spaces. For grouping, sorting, and querying, only the cleaned fields should be used. Fuzzy matching, spelling corrections, or subjective merging of names are not allowed.

2. Date of addition to the catalog, disposal deadline, and confirmation status of ransomware (20 minutes)

(1) Analyze the time when items were added to the directory

Create a sequence of monthly record counts from November 2021 to July 2026, sorted by dateAdded. Also, calculate the record counts for each year. The annual tables must indicate the first and last months covered by the data for that year, as well as the total number of months covered. Years 2021 and 2026 should be marked as incomplete years.

When comparing different years, at least one of the following comparable methods should be used:

- Only years that are completely covered in the attachments are compared.
- Compare the same periods from January to July across different years.
- All months are displayed directly, with clear indications that 2021 starts in November and 2026 ends on July 29th.

During the initial phase of directory establishment, existing vulnerabilities may be added to it. Therefore, the monthly and annual numbers of additions should be interpreted as part of the routine maintenance and updates of the KEV directory. They cannot be considered as the number of vulnerabilities disclosed, the number of attacks, or indicators of network attack trends.

(2) Calculating the term structure

Define the number of days for each record's directory disposal period:

$$D_i = \text{dueDate}_i - \text{dateAdded}_i$$

Deadline_days represents the number of calendar days between two dates, without counting the starting date as one day. The minimum, quartiles, median, mean, and maximum values for these durations are calculated to create a frequency table of all possible duration periods. The distribution of these durations is then analyzed based on the added_year parameter. This value represents the action window specified in the guidelines, not the actual time required for a particular organization to fix the issue. Nor should the severity of a vulnerability be determined solely based on its duration.

(3) Analyze the status of ransomware detection

Count the number and percentage of Known and Unknown entries separately, and create a cross-table based on added_year. The percentage of Known entries for a given year is defined as:

$$K_y = \frac{N_{y,\text{Known}}}{N_{y,\text{Known}} + N_{y,\text{Unknown}}}$$

The denominator here represents all records of individuals who joined KEV during that year. These values are stored as decimals and formatted as percentages when displayed. "Unknown" values cannot be considered negative evidence. Therefore, both the legend and the main text must use the official values or phrases like "Confirmed/Not Confirmed". They cannot be replaced with phrases like "Ransomware Vulnerability/Non-Ransomware Vulnerability". $K_y[0,1]$

3. Analysis of Manufacturer, Product, and Catalog Concentration (20 points)

(1) Create a summary table of manufacturers and products

Retain the original vendorProject and product fields. Use the vendor_clean and product_clean fields generated in step 1 for grouping. The cleaning process only removes leading and trailing spaces. No fuzzy matching is performed, and manufacturers with similar names are not merged arbitrarily. For each manufacturer, at least the following calculations are performed:

- Number of KEV records;
- Proportion of the total 1,656 records.
- Cumulative percentage;
- The amount of different product_clean

The manufacturers are sorted in a consistent manner based on “descending record count, ascending vendor_clean”. Then, they are grouped by “manufacturer + product”. A definitive sort must be performed first according to “descending record count, ascending vendor_clean, ascending product_clean” before using the head(30) function. It’s not allowed to directly extract the first 30 items from results where the order of ties isn’t specified. The “Multiple Products” in the attachment represent the original product names. These should be retained as a single text category and not split arbitrarily.

(2) Calculate concentration ratio

Let the proportion of records for each manufacturer be as indicated in the attachment. Calculate the concentration ratio for the top 5 and top 10 manufacturers: jp_j

$$CR_k = \sum_{j=1}^k p_j$$

Also calculate the Herfindahl-Hirschman index without multiplying by 10,000:

$$HHI = \sum_j p_j^2$$

The concentration level mentioned here refers only to how concentrated the KEV directory records are across various manufacturer-specific text labels. It does not represent market share concentration, nor can it be used to determine which manufacturers are less secure. Factors such as a manufacturer’s product scale, market coverage, and the way in which data is disclosed and recorded all influence the number of records.

4. CWE Multi-tag Statistics and Combined Query Functions (20 points)

(1) Properly expand the CWE list

In the attachments, 1,485 records contain at least one CWE. For 171 records, the list of CWEs is empty. Among the records with CWEs, some CVEs correspond to multiple pieces of code. A long table was created using the explode function or equivalent methods, with each row representing a “CVE-CWE” correspondence.

For each CWE, count the number of different CVEs associated with that code. Do not count the same CVE multiple times based on the number of lines of code. The proportion of CWEs that appear in records with a CWE identifier is defined as follows: c

$$S_c = \frac{\#\{\text{Different CVE with } c\}}{1485}$$

One CVE can contain multiple CWEs. Therefore, the sum of these values can exceed 1 (and exceed 100% when expressed as a percentage). It's not possible to force these values to be mutually exclusive. Among the records in the attachment, 294 are "Known" and contain at least one CWE, while 1,191 are "Unknown" and contain at least one CWE. When calculating the proportion of CWEs in each subset, 294 and 1,191 should be used as the denominators respectively. This should be clearly indicated in the table header or captions. It's not appropriate to divide by 1,485 overall, nor to divide by all 332 Known records or 1,324 Unknown records. S_c

(2) Implementing composite query functions

Implement the function:

```
filter_kev(df, start_date=None, end_date=None,
           vendor=None, ransomware=None, cwe=None)
```

Function requirements:

- The start and end dates are filtered using the dateAdded field as a closed interval.
- The vendor performs a case-insensitive substring match for vendor_clean.
- Ransomware only accepts the values "Known", "Unknown", or "None". Any invalid value should trigger an exception.
- First, convert CWE to uppercase and check against the pattern ^CWE-[0-9]+\$. Then, perform an exact match based on the list members. Fuzzy matching is not allowed in the resulting long string.
- When multiple non-empty conditions are provided at the same time, the logical "AND" operator is used.
- Do not modify the original DataFrame that was passed in.
- Return (result, summary): The result is a table sorted in descending order by dateAdded and ascending order by cveID. The summary includes at least the number of records, number of vendors, maximum date, and number of known issues. An empty result should still maintain the complete column structure, with the number of records, number of vendors, and number of known issues set to 0, and the maximum date left as a null value.

Design at least three sets of queries, covering different combinations of date criteria, vendor criteria, ransomware status, and CWE criteria. For each set, record the input parameters, number of rows returned, and a summary. Also, export the actual records; do not merely print the counts.

IV. Instructions for Optional Sections

1. KEV Directory Search and Visualization GUI

Design the program using Tkinter, PyQt, or other Python GUI libraries to directly read the course JSON data. To avoid mistaking date strings and semicolon-separated text in the CSV file for already parsed dates and lists, the GUI should not use kev_prepared.csv as input. At a minimum, implement the following:

1. Select a file and view the directory version, release date, number of records, and loading status.
2. Filter by addition date, manufacturer or project, product keywords, Known/Unknown, and CWE. Also includes a reset function.
3. At least two of the monthly record charts, vendor record charts, or CWE charts should be updated in response to filtering criteria.
4. The table displays CVE results. When selected, details such as a brief description, required actions, due date, and notes are displayed.
5. Export the current filtered results as a CSV file and the current graph as a PNG file.

Referenced official materials:

- CISA KEV Catalog: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- CISA JSON Schema:
https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities_schema.json
- KEV CC0 : <https://www.cisa.gov/sites/default/files/licenses/kev/license.txt>
- Pandas json_normalize: https://pandas.pydata.org/docs/reference/api/pandas.json_normalize.html
- Pandas explode: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.explode.html>
- Plotly Treemap: <https://plotly.com/python/treemaps/>

Question 4: Visualization of Campus Energy Consumption Data and Energy-Saving Analysis Tools

I. Problem Description

In college campuses, buildings such as classrooms, dormitories, laboratories, and libraries generate large amounts of data related to electricity consumption, water usage, and air conditioning operation on a daily basis. By collecting, cleaning, analyzing, and visualizing this data, administrators can understand the patterns of energy usage on campus, identify any abnormal energy consumption patterns, and develop recommendations for improving energy efficiency.

This task requires designing and implementing a “Campus Energy Consumption Visualization and Energy Savings Analysis Tool”. The program should be able to read Excel or CSV data files provided by course groups. It should analyze energy consumption data for different buildings, dates, and time periods, generate various graphical representations, and provide a preliminary assessment of any abnormal energy consumption patterns and potential for energy savings.

II. Experimental Requirements

1. [20 points] Use Pandas to read and organize campus energy consumption data files. Complete file reading, handling of missing values, handling of duplicate data, conversion of date fields, and organization of key fields.
2. 【20 points】 Conduct statistical analysis of the overall energy consumption on campus. Complete at least three of the following: total energy consumption per building, average energy consumption per building type, daily variations in total campus energy consumption, comparison between workdays and weekends, and ranking of energy consumption per unit area.
3. 【20 Points】 Create no less than 5 visualizations, including a bar chart showing the total electricity consumption of the building, a line chart depicting daily energy consumption, a pie chart showing the proportion of energy consumption by building type, a scatter plot showing the relationship between electricity consumption and air-conditioning operation time, a heat map showing energy consumption per unit area, and a chart marking days with abnormal energy consumption.
4. 【10 points】 Implement an abnormal energy consumption detection function. Methods such as $\text{mean} \pm 2$ standard deviations, interquartile range using box plots, or moving averages can be used to identify energy consumption records that are significantly higher or lower than normal. The results should be exported as Excel/CSV files.
5. 【10 Points】 Generate a written analysis based on the statistical results. The analysis should include, at a minimum, the entity with the highest energy consumption, the dates when energy consumption changed significantly, any abnormal records, and simple suggestions for energy savings.
6. [20 points] (Optional) Design a GUI interface that allows users to import data files, select buildings or date ranges, view statistical results and visualizations. The interface should also enable users to export analysis reports or images.

III. Instructions for Required Parts

- The data reading and cleaning process should not modify the original data files. Data cleaning must be performed within the program itself.

- “The overall energy consumption of a campus” should be analyzed from multiple perspectives, rather than just focusing on individual buildings.
- Graphs should have clear titles, labels for the horizontal and vertical axes, and a legend or color key.
- Abnormal energy consumption detection doesn’t require complex machine learning algorithms, but the detection rules must be clearly defined.
- The program should support saving statistical tables, exception records, and visualized images.

IV. Instructions for Optional Sections

- The GUI should support importing files in Excel/CSV format, selecting buildings, building types, and date ranges, previewing the results, and exporting the data.
- It’s also possible to implement an “energy-saving scoring” function. By using indicators such as energy consumption per unit area, the number of abnormalities, and the peak-valley difference, each building can be assigned an energy-saving risk level.
- Multiple charts can be combined into one analysis panel. For example, the filtering criteria can be displayed at the top, while line charts, bar charts, and exception records can be shown below.

V. Experimental Guidance

【Required Parts】

- Python libraries that may be used: Pandas, Matplotlib, Seaborn, NumPy.
- To calculate total energy consumption by building, the groupby() method can be used.
- For anomaly detection, the mean and standard deviation can be calculated first, and then records exceeding the threshold can be filtered out.

```
df = pd.read_excel("Campus Energy Consumption Data.xlsx")
building_sum = df.groupby("Building Name")[["Electricity Consumption", "Water Consumption"]].sum()
abnormal = df[df["Electricity Consumption"] > df["Electricity Consumption"].mean() + 2 * df["Electricity Consumption"].std()]
```

[Optional Section]

- The GUI can be implemented using tools like PyQt5, Tkinter, or Streamlit. Priority should be given to implementing functions for file import, parameter selection, result preview, and export.
- For optional features, the implementation details, core code, runtime screenshots, and functional outcomes should be described in the experiment report.

Question 5: Urban Air Quality Data Analysis and Pollution Early Warning Tools

I. Problem Description

Air quality is closely related to residents' health, urban transportation, industrial activities, and weather conditions. Air quality data typically includes indicators such as PM2.5, PM10, SO2, NO2, CO, O3, and AQI. By analyzing these data, it's possible to understand patterns in pollutant concentrations, compare air quality across different cities or monitoring stations, and issue early warnings during periods of poor air quality.

This task requires designing and implementing a "Urban Air Quality Data Analysis and Pollution Warning Tool". The program should be able to read air quality monitoring data, perform data cleaning, statistical analysis, graphical visualization, and pollution level determination, and generate analysis results.

II. Experimental Requirements

7. **【20 points】** Use Pandas to read air quality data files. Identify fields such as date, city, monitoring station, AQI, PM2.5, PM10, NO2, SO2, CO, and O3. Handle missing and abnormal values.
8. **【15 points】** Complete the basic statistics on air quality. Calculate at least the average AQI, highest AQI, number of polluted days, and percentage of days with good air quality for each city or monitoring station. Export the results as an Excel file.
9. [15 points] Complete the analysis of pollutant correlations. Calculate the correlation coefficients between PM2.5, PM10, NO2, SO2, CO, O3, and AQI. Create correlation heatmaps.
10. **【20 Points】** Create at least 5 graphs, including line charts showing AQI changes over time, bar charts showing average AQI values for different cities, pie charts showing the proportion of various pollution levels, heat maps showing correlations between pollutants, scatter plots of PM2.5 and AQI, and box plots showing monthly average AQI values.
11. [10 points] Implement a simple pollution warning system that displays the date of pollution, the city or location, the main pollutants, and the pollution level based on AQI standards.
12. [20 points] (Optional) Design a GUI or interactive visualization page that allows users to select cities, date ranges, and pollution indicators. The system should dynamically display graphs and pollution warning results based on the user's selections.

III. Instructions for Required Parts

- When cleaning data, pay attention to empty strings, invalid values, negative numbers, and special symbols.
- To determine the level of pollution, you can refer to the common AQI categories: 0-50 is considered excellent, 51-100 is good, 101-150 is mild pollution, 151-200 is moderate pollution, 201-300 is severe pollution, and above 300 is extremely severe pollution.
- Correlation analysis should use numerical pollutant indicators. Non-numerical fields need to be converted first.
- The graph should show the comparative relationships between cities, months, or different pollutants.
- The program should ultimately save the cleaned data tables, statistical tables, pollution warning results, and no fewer than 5 images.

IV. Instructions for Optional Sections

- The GUI or interactive interface should support importing data, selecting cities or locations, choosing date ranges, selecting pollutant indicators, viewing graphical representations, and viewing alert results.
- Pyecharts can be used to create interactive line charts, maps, or timeline charts.
- A simple prediction function can be designed, such as using the moving average of AQI values over the past 7 days to predict air quality trends for the next day.

V. Experimental Guidance

【Required Parts】

- Python libraries that may be used: Pandas, Matplotlib, Seaborn, NumPy, OpenPyXL.
- The date fields can be converted using `pd.to_datetime()`.
- Correlation heatmaps can be created using `seaborn.heatmap()`.

```
df["日期"] = pd.to_datetime(df["日期"])
```

```
city_aqi = df.groupby("城市")["AQI"].agg(["mean", "max", "count"])
```

```
corr = df[["AQI", "PM2.5", "PM10", "NO2", "SO2", "CO", "O3"]].corr()
```

[Optional Section]

- The GUI can be implemented using tools like PyQt5, Tkinter, or Streamlit. Priority should be given to implementing functions for file import, parameter selection, result preview, and export.
- For optional features, the implementation details, core code, runtime screenshots, and functional outcomes should be described in the experiment report.

Question 6: Tools for Organizing University Library Borrowing Data and Analyzing Reading Behaviors

I. Problem Description

University libraries keep extensive records of book borrowing information, including book titles, categories, borrowing and return dates, reader types, colleges, and grades. By analyzing this data, it's possible to understand students' reading preferences, popular book categories, borrowing patterns across different colleges, and overall utilization of library resources.

This task requires designing and implementing a “tool for organizing university library borrowing data and analyzing reading behaviors”. The program should be able to read library borrowing data, perform data cleaning, statistical analysis, and visualizations. It should also analyze which books are most popular and identify instances of overdue returns.

II. Experimental Requirements

13. **【20 points】** Use Pandas to read the book borrowing data file. Handle missing values, duplicate borrowing records, and date format errors. Calculate the borrowing duration for each record.
14. **【15 points】** Complete the overall statistics on book borrowing. At a minimum, calculate the total number of borrowings, the number of borrowings for different book categories, the number of borrowings by month, the number of borrowings by faculty, and the average borrowing duration.
15. **【15 points】** Complete an analysis of popular books and categories. Output the top 10 books in terms of borrowing frequency, the top 5 book categories. Also, determine the most frequently borrowed book categories across different colleges.
16. **【20 Points】** Create no less than 5 visualizations, including a line chart showing monthly borrowing volumes, a bar chart showing borrowing volumes by book category, a horizontal bar chart showing popular books, a comparison chart of borrowing volumes across colleges, a histogram showing borrowing duration distribution, and a heat map showing borrowing patterns by college and category.
17. **【10 points】** Implement overdue return analysis. Assume that regular books can be borrowed for 30 days. If the borrowing period exceeds 30 days, it's considered overdue. The system should count the number of overdue records, calculate the overdue rate, identify the colleges or reader types with the most overdue cases, and generate a report of overdue records.
18. **[20 points]** (Optional) Design a GUI that allows users to import borrowing data, select colleges, months, or book categories, and view corresponding statistical results and charts. Alternatively, implement a simple book recommendation feature.

III. Instructions for Required Parts

- The borrowing period is calculated based on the “Borrowing Date” and the “Return Date”.
- There may be inconsistencies in the names of book categories. These should be merged into unified categories based on actual circumstances.
- Popular books analysis should be presented in tabular form, with at least one graph included.
- The college-category heatmap can reflect the reading preferences of different colleges.
- There should be clear rules for determining overdue situations, and these rules should be explained in the experimental report.

IV. Instructions for Optional Sections

- The GUI should support data file selection, filtering by college/month/category, display of popular books, graphical previews, and result export.
- The simple book recommendation feature can be implemented based on popular categories in the academy, categories selected by users, or recently popular books.
- The “Reading Profile” feature can be added, showing the total number of loans taken by a particular faculty, the most frequently borrowed categories, the average borrowing duration, and the overdue rate.

V. Experimental Guidance

【Required Parts】

- Python libraries that may be used: Pandas, Matplotlib, Seaborn, NumPy, OpenPyXL.
- The borrowing duration can be calculated by subtracting one date field from another.
- The relationship between colleges and categories can be analyzed using pivot tables.

```
df["borrow_date"] = pd.to_datetime(df["return_date"])
```

```
df["time"] = (df["return_date"] - df["borrow_date"]).dt.days
```

```
top_books = df["book_name"].value_counts().head(10)
```

[Optional Section]

- The GUI can be implemented using tools like PyQt5, Tkinter, or Streamlit. Priority should be given to implementing functions for file import, parameter selection, result preview, and export.
- For optional features, the implementation details, core code, runtime screenshots, and functional outcomes should be described in the experiment report.

Question 7: Analysis and Visual Assessment of Urban Air Quality Monitoring Data

I. Problem Description

With the rapid development of industrialization and urbanization, air pollution has become a major environmental issue worldwide. Accurate monitoring and analysis of air quality data are crucial for assessing pollution levels, identifying sources of pollution, and formulating environmental policies. This task requires you to use the Python programming language to process, analyze, and visualize air quality data from one or multiple cities. You should create a tool for analyzing and evaluating air quality data.

II. Experimental Requirements

【Required Section】 (80 points)

1. Data Reading and Preprocessing (15 points)

Use the Pandas library to read the provided air quality dataset (in CSV or Excel format), which includes fields for date, monitoring station, AQI (Air Quality Index), and concentrations of pollutants such as PM2.5, PM10, SO₂, NO₂, CO, and O₃.

Clean the data:

Handle missing values (e.g., by forward filling, mean filling, or deleting missing records).

Handle outliers (e.g., identify and mark data that exceeds reasonable ranges).

Convert the date fields to standard time format and set them as the index of the DataFrame.

Group the data by monitoring stations or cities, and calculate the daily/weekly/monthly statistics for each pollutant's concentration (mean, maximum, minimum, standard deviation).

2. Single-site Time Series Analysis (20 points)

For at least 3 different monitoring sites, create the following graphs for each site (at least 3 graphs per site):

AQI Time Series Line Chart: Displays the trend of AQI values at that site over a certain period of time.

Line charts of major pollutant concentrations: Plot the concentration trends of six pollutants—PM2.5, PM10, SO₂, NO₂, CO, and O₃—in the same chart, using different colors to distinguish between them.

Box plot of pollutant concentrations: Displays the concentration distribution of various pollutants at this site.

Correlation Heatmap: Calculates the correlation coefficients between various pollutants on the site, and displays them in a heatmap format.

3. Multi-site Comparative Analysis (20 points)

Select at least 5 monitoring sites for cross-comparative analysis:

AQI Bar Chart Comparison: Displays bar charts showing the average AQI at each monitoring station over a specific time period.

Pollutant Radar Charts: Radar charts are created for each monitoring station, showing the concentration levels of six major pollutants. This makes it easy to compare the pollution patterns of different stations visually.

Spatial Distribution Scatter Plot: Using the latitude and longitude information of each station, a scatter plot of the average AQI values for each station is created on a map background (basemap, folium, or pyecharts can be used for this purpose). The size or color of the dots represents the level of AQI. If the data does not contain latitude and longitude information, these values can be manually mapped using the station names (the mapping method must be specified in the report), or text matching can be performed using the station names.

Time Trend Comparison Line Chart: Displays line charts of AQI time series for multiple stations on the same graph, allowing comparison of pollution trends across different stations.

4. Air Quality Rating Assessment (10 points)

According to the air quality classification standards set by national authorities or the World Health Organization (WHO) (e.g., AQI 0-50 indicates excellent air quality, 51-100 indicates good air quality, 101-150 indicates mild pollution, etc.), the daily AQI values are categorized accordingly.

Calculate and visualize the distribution of days per air quality level for each station (using stacked bar charts or pie charts).

Calculate and output the “proportion of days with good air quality” for each station.

5. Data Export and Report Generation (15 points)

Export all the analysis results mentioned above (statistical tables, charts). Charts should be exported in PNG/JPG format, while statistical tables should be exported in Excel or CSV format.

Generate a brief text report (in TXT or Markdown format) that summarizes key findings regarding air quality at each site, major pollutants, and the times of day when pollution is most/severely lowest.

[Optional Section] (20 points: Choose one or both of the following options)

Direction A: GUI Interaction Interface Design

Design a graphical user interface using PyQt5 or Tkinter that allows users to:

Import the data file through the file selection dialog box.

Select the monitoring site and time range you want to analyze from the drop-down menu.

Click the button to generate and preview various graphics from the required sections above.

Display key statistical information on the interface (such as average AQI, percentage of days with good air quality, etc.).

Direction B: Pollutant Concentration Prediction

Use machine learning methods (such as linear regression, decision trees, random forests, or LSTM) to make short-term predictions for PM2.5 or AQI values at a specific location (e.g., predicting values for the next 3/7 days).

Use historical data as the training set, split it into a training set and a test set, and evaluate the model's performance using metrics such as MAE and RMSE.

Compare and visualize the predicted results with the actual values (use a line chart to display the relationship between predicted and true values).

III. Instructions for Required Parts

Dataset Format Notes: The provided CSV file must contain the following fields: date, station (name of the monitoring station), station_lon (longitude, optional), station_lat (latitude, optional), AQI, PM2.5, PM10, SO2, NO2, CO, O3. If the data does not include latitude and longitude information, these can be manually mapped using the station names, with the mapping method specified in the report.

Drawing requirements: All graphs must include titles, axis labels, and legends (where applicable). The graphs should be clear and aesthetically pleasing. It is recommended to use the matplotlib and seaborn libraries for drawing.

Time processing: Use pandas' datetime functions to handle time series data. Resample() and rolling() methods can be utilized for resampling and rolling calculations.

Correlation analysis: The correlation coefficient matrix is calculated using df.corr(), and visualized with seaborn.heatmap().

IV. Instructions for Optional Sections

GUI Design: The interface should be simple, aesthetically pleasing, and well-organized. Recommended GUI libraries are PyQt5 or Tkinter. You can refer to the official PyQt5 documentation or relevant tutorials for more details.

Machine Learning Predictions: It's recommended to use models from the scikit-learn library. The data should be standardized, and a training set and test set should be created. If deep learning techniques like LSTM are to be used, TensorFlow or PyTorch can be considered.

V. Experimental Guidance

Pandas for reading CSV files: `df = pd.read_csv('data.csv', parse_dates=['date'])` Handling missing values: `df.fillna(method='ffill', inplace=True)` or `df.dropna(inplace=True)` Grouping and aggregation: `df.groupby('station')['PM2.5'].mean()` Resampling: `df.resample('W').mean()` (resample by week) Creating heatmaps: `sns.heatmap(df.corr(), annot=True, cmap='coolwarm')` Creating radar charts: Use the polar projection: `projection='polar'` Map visualization: It's recommended to use pyecharts' Map or Geo components.

VI. Data Acquisition Guidelines

Recommended data sources:

China National Environmental Monitoring Centre: <http://www.cnemc.cn/> Air quality data from the U.S.

Embassy in China: <https://www.stateair.net/> Kaggle air quality dataset: <https://www.kaggle.com/datasets>

(search for "Air Quality") China Air Quality Online Monitoring and Analysis Platform:

<https://www.aqistudy.cn/>

Question 8: Analysis and Prediction of Spatiotemporal Characteristics of Shared Bicycle Travel Demand

I. Problem Description

Shared bicycles have become an important means of short-distance transportation in cities. Understanding the spatio-temporal patterns of shared bicycle usage is crucial for optimizing vehicle scheduling, improving operational efficiency, and enhancing user experience. This task requires you to use the Python programming language to conduct in-depth analysis of historical data from a shared bicycle platform. You should identify the spatio-temporal patterns of demand and attempt to develop predictive models for future demand.

II. Experimental Requirements

【Required Section】 (80 points)

1. Data Reading and Preprocessing (15 points)

Use Pandas to read the shared bike riding data provided (in CSV format). The data includes fields such as start time, end time, starting station ID/name, starting station's latitude and longitude, ending station ID/name, ending station's latitude and longitude, user type, and riding duration.

Clean the data:

Handle missing values (e.g., delete records with missing key fields).

Handle outliers (e.g., records with negative or excessively long riding durations, or records with significantly incorrect latitude and longitude values).

Convert the time field to the standard time format.

Derived new features:

Extract information such as the hour, day of the week, month, and whether it's a workday when the ride starts.

Calculate the riding distance (if latitude and longitude are provided, the Haversine formula can be used for estimation).

Divide into traffic periods (such as morning rush hour, evening rush hour, off-peak hours, and nighttime).

2. Time-based Analysis (20 points)

Create a line chart showing the total riding distance over time (data can be aggregated by hour, day, week, month, etc.).

Analyze and visualize the distribution of cycling activity across different time periods (hours). Present the 24-hour average cycling volume as a bar chart.

Analyze and visualize the comparison of cycling volumes on weekdays and weekends (using box plots or grouped bar charts).

Analyze and visualize the differences in riding behaviors among different user types (such as regular users and premium members), including metrics like riding duration and usage frequency.

3. Spatial Dimension Analysis (20 points)

Identify the most popular starting and ending stations (ranked by number of rides). Display the top 10 stations along with their locations.

Create a heat map of ride volumes at each station: Display the distribution of riders departing from or arriving at each station as a heat map on the map.

Analyze OD (origin-destination) traffic between sites: Select the top 10 most popular sites, identify the 20 pairs of ODs with the highest traffic, and create a Sankey diagram or chord diagram to visualize the traffic flow between them. Ensure that the graphical information remains reasonably dense.

Identifying high-demand areas: Divide the city into grids, count the number of departures/arrivals in each grid, and create a grid density map.

4. Travel Demand Forecasting (15 points)

Predicting travel demand for a future period (e.g., the next hour) based on historical data, on a per-site or per-region basis.

Try at least two different prediction methods (such as the time-series model ARIMA, machine learning models XGBoost/LightGBM, or a simple historical mean method), and compare their prediction accuracy.

Use appropriate evaluation metrics (such as MAE, RMSE, MAPE) to assess model performance.

5. Result Visualization and Reporting (10 points)

Export the above analysis results (statistical tables, charts).

Prepare an analysis report that summarizes the spatio-temporal patterns of shared bike usage and user behavior characteristics. Also, analyze and discuss the predicted results.

[Optional Section] (20 points: Choose one or both of the following options)

Direction A: Interactive visualization dashboards

Build an interactive data dashboard using Plotly Dash, pyecharts, or Streamlit.

The dashboard should include at least the following:

Time range selector (allows dynamic filtering of data time ranges).

Map component (shows site distribution and riding volume).

Dynamic charts (trend charts showing changes in riding volume over time, station rankings, etc.).

Direction B: Dynamic Scheduling Strategy Simulation

Based on the analysis results, design a simple vehicle dynamic scheduling strategy (e.g., scheduling vehicles to high-demand areas in advance based on real-time demand forecasts).

Use simulated or historical data to backtest the effectiveness of this strategy (e.g., by calculating metrics such as improved vehicle supply-demand balance and reduced idling rates after scheduling).

III. Instructions for Required Parts

Dataset format notes: The provided CSV file must contain the following fields: start_time (start time), end_time (end time), start_station_id, start_station_name, start_lat, start_lon, end_station_id,

end_station_name, end_lat, end_lon, user_type (user type), and trip_duration (riding duration in seconds). The time format is YYYY-MM-DD HH:MM:SS.

Time processing: Utilize pandas' datetime functionality. To extract the hour: `df['hour'] = df['start_time'].dt.hour`.

Geospatial analysis: Geopy can be used to calculate distances, while folium or pyecharts can be utilized for map visualization.

OD Analysis: The networkx library can be used to create graphs depicting traffic patterns between sites.

Predictive modeling: It's recommended to use the scikit-learn, statsmodels, or XGBoost libraries.

IV. Instructions for Optional Sections

Interactive Dashboards: Plotly Dash and Streamlit are popular frameworks for building data-driven applications, with relatively gentle learning curves.

Scheduling strategy simulation: This section focuses on conceptual design and logical implementation, without requiring actual deployment.

V. Experimental Guidance

读取数据并解析时间: `df = pd.read_csv('data.csv', parse_dates=['start_time', 'end_time'])`

提取时间特征: `df['hour'] = df['start_time'].dt.hour`、`df['weekday'] = df['start_time'].dt.weekday`

分组聚合: `hourly_counts = df.groupby('hour').size()`

Haversine 距离计算（示例函数）：

python

`from math import radians, sin, cos, sqrt, asin`

`def haversine(lon1, lat1, lon2, lat2):`

`R = 6371 # Earth's radius in kilometers` `dlon = radians(lon2 - lon1)` `dlat = radians(lat2 - lat1)` `a =`
`sin(dlat/2)**2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon/2)**2` `return 2 * R * asin(sqrt(a))` To
create a heat map: Use folium.plugins.HeatMap or pyecharts.HeatMap.

VI. Data Acquisition Guidelines

Recommended data sources:

Citi Bike system data (New York): <https://www.citibikenyc.com/system-data> Capital Bikeshare data (Washington): <https://www.capitalbikeshare.com/system-data> Kaggle shared bike demand prediction dataset: <https://www.kaggle.com/datasets?search=bike+sharing> Public data from domestic shared bike platforms (available through APIs or public sources)

Question 9: Social Media Public Opinion Analysis and Emotional Evolution Tracking

I. Problem Description

Social media has become an important platform for the public to express their opinions and spread information. By analyzing social media data from platforms like Weibo and Twitter, it's possible to understand public sentiment, track trending events, and assess brand reputation. This assignment requires you to use the Python programming language to perform sentiment analysis, topic modeling, and temporal trend analysis on the provided social media text dataset. You'll also need to develop a tool for analyzing and visualizing this data.

II. Experimental Requirements

【Required Section】 (80 points)

1. Data Reading and Preprocessing (15 points)

Use Pandas to read the provided social media data in CSV or JSON format, which includes fields such as text content, posting time, user ID, number of likes, number of comments, and number of shares.

Preprocessing of text data:

Remove URLs, @mentions, and #hashtags (but retain hashtags as features).

Remove special characters, punctuation marks, and numbers (optional).

Perform word segmentation (jieba for Chinese, nltk or spaCy for English).

Remove stopwords (stopword lists in both Chinese and English are provided).

Convert the time fields to the standard time format. The format of the created_at field is YYYY-MM-DD HH:MM:SS (e.g., 2024-01-15 14:30:00).

2. Emotional Analysis (20 points)

Use sentiment dictionaries or pre-trained models to determine the sentiment polarity of each text (positive/negative/neutral) and calculate a sentiment score (a continuous value between -1 and 1).

It is recommended to use lightweight solutions:

In Chinese, SnowNLP is recommended (it's based on a naive Bayesian sentiment dictionary).

For English processing, TextBlob or VADER are recommended.

If hardware conditions permit (it's recommended to use a GPU), you can try using pre-trained BERT models (such as those available in the transformers library) to improve the accuracy of sentiment analysis.

Statistics and visualization:

Sentiment Distribution Pie Chart: Displays the proportion of positive, negative, and neutral texts.

Emotional Score Distribution Histogram: Displays the overall distribution of emotional scores.

Time-series sentiment trend line chart: Aggregated by day/week, showing the trend of average sentiment scores over time.

Top Keywords Word Clouds: Generate word clouds for positive and negative texts respectively, and analyze the differences in keywords across different emotional contexts.

3. Topic and Event Tracking (20 points)

Using TF-IDF or LDA topic models, extract at least 5 main topics from the text, and provide the Top 10 keywords for each topic.

Track how the popularity of each topic changes over time (create line charts showing the variation in discussion volume for each topic over time).

Identifying trending topics: By detecting sudden increases in discussion volume, this feature highlights topics or events whose popularity has risen sharply in a short period of time.

Analyze the relationship between topics and emotions: Calculate the average emotional score of texts under each topic, and identify “positive topics” and “negative topics”.

4. User Influence Analysis (10 points)

Based on metrics such as likes, comments, and shares, the user’s influence score is calculated comprehensively (with customizable weighting formulas).

Identify and output the Top 10 most influential users, along with their basic information and influence scores.

Visualize the distribution of user influence (e.g., by creating box plots or histograms of influence scores).

5. Result Export and Reporting (15 points)

Export all the above analysis results (statistical tables, charts).

Generate an analysis report in PDF or Markdown format, summarizing public opinion trends, emotional evolution, key topics, and important users.

[Optional Section] (20 points: Choose one or both of the following options)

Direction A: GUI Interaction Interface Design

Design a graphical user interface using PyQt5 or Tkinter that allows users to:

Import the data file.

Select the time range for the analysis.

Clicking the buttons generates visualizations for sentiment analysis, topic analysis, user analysis, and other modules.

Display key statistical information on the interface.

Direction B: Event Context Construction

For a specific trending event (relevant text must be included in the dataset), the development of the event is automatically reconstructed based on chronological order and text similarity. In other words, key moments and representative texts are organized along a timeline.

Use text summarization techniques (such as TextRank) to generate brief summaries for each key node.

Visualize the sequence of events in the form of a timeline or story map.

III. Instructions for Required Parts

Dataset format details: The provided CSV/JSON files must contain the following fields at a minimum: text (content), created_at (publication time, in YYYY-MM-DD HH:MM:SS format), user_id (user ID), likes (number of likes), comments (number of comments), retweets (number of retweets). It is recommended that the dataset contain no less than 5000 entries.

Chinese word segmentation: The jieba library is used for word segmentation. For English, nltk.word_tokenize() can be used.

Emotion Analysis Tools:

In Chinese, SnowNLP is recommended (it's based on a naive Bayesian sentiment dictionary).

For English processing, TextBlob or VADER are recommended.

If capable and the hardware conditions permit, the pre-trained BERT model can be used for more accurate sentiment analysis.

Topic Modeling: The LDA topic model is implemented using the LdaModel class from the gensim library.

Word Cloud: Generates word cloud diagrams using the wordcloud library (Chinese font configuration required).

IV. Instructions for Optional Sections

Event context construction: Relevant algorithms for constructing news event contexts can be referenced (such as text clustering and summarization based on timelines).

Text Summary: You can use gensim.summarization (TextRank algorithm) or the pre-trained summarization models from the transformers library.

V. Experimental Guidance

Pandas reads JSON: df = pd.read_json('data.json', lines=True) Chinese word segmentation and stopword removal:

```
python
```

```
import jieba
```

```
stopwords = set([line.strip() for line in open('stopwords.txt', 'r', encoding='utf-8')])
```

```
df['tokens'] = df['text'].apply(lambda x: [w for w in jieba.cut(x) if w not in stopwords and len(w) > 1])
```

SnowNLP:

```
python
```

```
from snownlp import SnowNLP
```

```
df['sentiment_score'] = df['text'].apply(lambda x: SnowNLP(x).sentiments)
```

LDA:

```
python
```

```
from gensim import corpora, models
```

```
dictionary = corpora.Dictionary(df['tokens'])
corpus = [dictionary.doc2bow(tokens) for tokens in df['tokens']]
lda_model = models.LdaModel(corpus, num_topics=5, id2word=dictionary, passes=10)
words map:
python
from wordcloud import WordCloud
wordcloud = WordCloud(font_path='simhei.ttf', background_color='white').generate(' '.join(all_tokens))
Time series resampling: df.resample('W', on='created_at')['sentiment_score'].mean()
```

VI. Data Acquisition Guidelines

Recommended data sources:

Kaggle Social Media Sentiment Analysis Dataset: <https://www.kaggle.com/datasets?search=twitter+sentiment>
Data collected using Weibo's public API (it's necessary to specify the data collection method and time range).
Twitter Public Dataset: <https://www.kaggle.com/datasets?search=twitter> Chinese social media corpora (e.g., THUOCL, Sina Weibo public datasets)

Hint:

Some questions do not restrict the dataset, and you are encouraged to search for datasets independently under the requirements of the question, complete the data cleaning and analysis, and design a GUI. Note that the dataset you choose should be at least of a certain size and not too small, otherwise it will not be considered a valid assignment.

